Lehrerexemplar · Boss-Kampf 2 · Welt 2

Erwartungshorizont: Das Turnier der Bogenschützen

Erwartungshorizont / Bewertungsbogen für _____

I, II, III: erreichbare Bewertungseinheiten im jeweiligen Anforderungsbereich · BE: vom Prüfling erreichte Bewertungseinheiten

Erwartete Prüfungsleistungen	I	II	III	BE
a) Dokumentation des Sortiervorgangs Kernpunkte: Belegung nach jedem äußeren Durchlauf (i = 1 bis 5) mit Namen und Ringzahl; unveränderte Zeile bei i = 2; Tom wandert bei i = 3 bis an Index 0; Lea wird bei i = 4 direkt hinter Ben eingefügt; Endzustand Tom, Ben, Lea, Finn, Ida, Mila.	2	2		
b) Angabe des Verfahrens; Erläuterung der Funktionsweise; Begründung Kernpunkte: Sortieren durch Einfügen (Insertionsort); linker Teil teilnehmer[0] bis teilnehmer[i − 1] ist bereits sortiert; teilnehmer[i] wird durch Vertauschen mit dem linken Nachbarn so weit nach links bewegt, bis j = 0 ist oder der linke Nachbar mindestens gleich viele Ringe hat; durch den echten Vergleich < wird bei gleicher Ringzahl nicht getauscht – Ben bleibt vor Lea (stabiles Verfahren).	1	2		
c) Bestimmung der Vergleichsanzahlen; Beurteilung der Aussage Kernpunkte: 11 Vergleiche bei Abbildung 3; ungünstigster Fall bei aufsteigend sortierten Ringzahlen (z. B. 40, 45, 50, 55, 60, 65) mit 1 + 2 + 3 + 4 + 5 = 15 Vergleichen; allgemein n(n − 1)/2; bei Verdopplung etwa vierfache Vergleichszahl (499 500 → 1 999 000), also eher 2 s statt 1 s – Aussage falsch. Speicherbedarf: konstant (Tausch innerhalb der Reihung).	1	2	1	
d) Implementierung der Operation nachtragen Kernpunkte: Abfrage s == null oder anzahl == teilnehmer.length mit Rückgabe false vor jeder Veränderung; Verschieben von hinten beginnend bei Index anzahl (teilnehmer[i] = teilnehmer[i − 1]); Schleife endet bei i == 0 oder wenn der linke Nachbar mindestens gleich viele Ringe hat (echter Vergleich <, dadurch hinter Gleichstand); i > 0 als erste Teilbedingung (kein Zugriff auf teilnehmer[−1]); Einsetzen von s, anzahl erhöhen, true zurückgeben.	1	2	2	
e) Dokumentation des Werteverlaufs; Analyse der maximalen Durchläufe Kernpunkte: suche(nummern, 42): (0, 14, 7), (8, 14, 11), (8, 10, 9), Rückgabe 9; suche(nummern, 20): (0, 14, 7), (0, 6, 3), (4, 6, 5), (4, 4, 4), Rückgabe −1; jeder Durchlauf halbiert den Suchbereich mindestens (100 → 50 → 25 → 12 → 6 → 3 → 1), daher höchstens 7 Durchläufe ($2^6 = 64 \le 100 < 128 = 2^7$); bei Verdopplung höchstens ein Durchlauf mehr (200 → 8).	1	3		
f) Analyse des fehlerhaften Struktogramms Kernpunkte: Fehler 1: Startwert bester ← teilnehmer[0] ohne Prüfung des Vereins; Fehler 2: Zählschleife bis anzahl statt anzahl − 1; Fehler 3: ≥ statt >; Auswirkungen: vereinsfremder Schütze bzw. kein null bei fehlendem Verein; Zugriff auf teilnehmer[anzahl] (null) → Laufzeitfehler bei jedem Aufruf; bei Gleichstand wird der letzte statt der erste geliefert; korrigiertes Struktogramm mit bester ← null, Schleife 0 bis anzahl − 1, Bedingung „bester = null oder … >“.	1	2	1	
Summe	7	13	4	**/ 24**

AFB-Verteilung: I 7 BE (29 %) · II 13 BE (54 %) · III 4 BE (17 %) – Abitur-Richtwert 30 / 53 / 17 %. Zeitbudget: 24 BE · 2,5 min = 60 min.

Die gewählten Lösungsansätze und -wege müssen nicht mit den dargestellten Lösungen identisch sein. Fachlich richtige Alternativen sind unter Berücksichtigung der verbindlichen Vorgaben (EH 2025) entsprechend zu bewerten.

Rückmeldung zum Boss-Kampf (fließt nicht in die Note ein)

- Auswertung durch die Lehrkraft oder als Selbst-/Peer-Korrektur nach den Kernpunkten dieses Erwartungshorizonts.
- Papier-Compiler zu d): Handschrift unverändert abgetippt, alle 8 Tests grün beim ersten Versuch → **Papier-Compiler bestanden**; sonst Fehlerlogbuch und Korrektur in der IDE.
- Fallen-Check: In d) und f) sind alle Leer-, Index- und null-Abfragen korrekt (d: null/voll vor jeder Veränderung, i > 0 vor dem Zugriff auf teilnehmer[i − 1]; f: Grenze anzahl − 1, Prüfung bester = null vor bester.getRinge()).
- Orientierung (informativ, Abi-Skala): ab 95 % 15 P · 85 % 13 P · 75 % 11 P · 65 % 9 P · 55 % 7 P · 45 % 5 P · 40 % 4 P · 27 % 2 P. Für 24 BE: 23 BE ≈ 15 P, 18 BE ≈ 11 P, 10 BE ≈ 4 P.
- Zusätzlich im Lab: Online-Boss **„Das Wettfischen am Drachensee“** (25 min, automatische Auswertung, bester Versuch zählt). Die Ergebnisse sieht die Lehrkraft im Lab unter Kurs → Auswertung (Boss-Ergebnisse).

Musterlösungen, Kernpunkte und typische Fehler

a) Dokumentation des Sortiervorgangs 4 BE (I 2 · II 2 · III 0)

Musterlösung

Belegung nach jedem Durchlauf der äußeren Schleife (geprüft durch Ausführen einer instrumentierten Java-Fassung von Abbildung 2):

nach Durchlauf	[0]	[1]	[2]	[3]	[4]	[5]
Start	Ida 47	Ben 52	Mila 45	Tom 58	Lea 52	Finn 50
i = 1	Ben 52	Ida 47	Mila 45	Tom 58	Lea 52	Finn 50
i = 2	Ben 52	Ida 47	Mila 45	Tom 58	Lea 52	Finn 50
i = 3	Tom 58	Ben 52	Ida 47	Mila 45	Lea 52	Finn 50
i = 4	Tom 58	Ben 52	Lea 52	Ida 47	Mila 45	Finn 50
i = 5	Tom 58	Ben 52	Lea 52	Finn 50	Ida 47	Mila 45

Vertauschungen je Durchlauf: 1 · 0 · 3 · 2 · 2 (zusammen 8); Ringzahl-Vergleiche: 1 · 1 · 3 · 3 · 3 (zusammen 11, vgl. c).

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Zeilen i = 1 und i = 2 korrekt (bei i = 2 keine Veränderung, da 47 < 45 falsch)	I	1	☐
Zeile i = 3: Tom wird schrittweise bis an Index 0 getauscht	I	1	☐
Zeile i = 4: Lea wird nur bis hinter Ben getauscht (Abbruch bei 52 < 52)	II	1	☐
Zeile i = 5: korrekter Endzustand Tom, Ben, Lea, Finn, Ida, Mila	II	1	☐

Alternativen / Teilpunkte: Folgefehler: Ist eine Zeile falsch, die Verarbeitung danach aber folgerichtig fortgesetzt, entfällt nur der Kernpunkt der fehlerhaften Zeile. Die Angabe nur der Ringzahlen ist nicht ausreichend, da Ben und Lea in Zeile i = 4 unterscheidbar sein müssen.

Typische Fehler

- Zeile i = 2 wird weggelassen, weil sich nichts ändert.
- Es wird der Zustand nach jedem einzelnen Tausch statt nach jedem äußeren Durchlauf notiert.
- Lea wird bei i = 4 vor Ben gestellt (Vergleich wie ≤ gelesen).
- Bei i = 3 wird nach dem ersten Tausch abgebrochen (Tom landet an Index 2).

b) Angabe des Verfahrens; Erläuterung der Funktionsweise; Begründung 3 BE (I 1 · II 2 · III 0)

Musterlösung

Es handelt sich um Sortieren durch Einfügen (Insertionsort).

Vor dem Durchlauf mit dem Wert i ist der Bereich teilnehmer[0] bis teilnehmer[i − 1] bereits absteigend sortiert. Der Schütze an Index i wird in diesen Bereich eingefügt: Solange er nicht ganz vorne steht (j > 0) und sein linker Nachbar weniger Ringe hat, werden beide vertauscht und j wird um 1 verringert. Die innere Schleife endet, sobald der linke Nachbar mindestens gleich viele Ringe hat oder Index 0 erreicht ist. Danach ist teilnehmer[0] bis teilnehmer[i] sortiert; nach dem letzten Durchlauf (i = anzahl − 1) die gesamte Reihung.

Ben steht anfangs an Index 1, Lea an Index 4. Wird Lea bei i = 4 nach links getauscht, ist bei Erreichen von Ben die Bedingung 52 < 52 falsch – die innere Schleife bricht ab. Ein Schütze wird also nie an einem Schützen mit gleicher Ringzahl vorbeigetauscht; die ursprüngliche Reihenfolge gleich guter Schützen bleibt erhalten (das Verfahren ist stabil).

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Angabe: Sortieren durch Einfügen (Insertionsort)	I	1	☐
Funktionsweise: sortierter linker Bereich, Einfügen durch Nachbartausch, Abbruchbedingungen	II	1	☐
Begründung über den echten Vergleich < (kein Tausch bei Gleichstand)	II	1	☐

Alternativen / Teilpunkte: „Einfügesortierung“ ist gleichwertig. Wird ein falsches Verfahren genannt, die Funktionsweise aber zutreffend beschrieben, wird der Kernpunkt zu II vergeben. Der Begriff „stabil“ ist nicht gefordert.

Typische Fehler

- Verwechslung mit Bubblesort wegen der Nachbartausche.
- Begründung „Ben stand vorher vorne“ ohne Bezug auf die Vergleichsbedingung.
- Funktionsweise nur als Nacherzählung des Struktogramms ohne Idee des sortierten Bereichs.

c) Bestimmung der Vergleichsanzahlen; Beurteilung der Aussage 4 BE (I 1 · II 2 · III 1)

Musterlösung

Abbildung 3: Durchlauf i = 1: 1 Vergleich (dann j = 0), i = 2: 1, i = 3: 3 (dann j = 0), i = 4: 3, i = 5: 3 – zusammen **11 Ringzahl-Vergleiche**.

Ungünstigster Fall: Die Ringzahlen sind streng aufsteigend sortiert, z. B. 40, 45, 50, 55, 60, 65. Dann muss jeder Schütze ganz nach vorne getauscht werden; im Durchlauf i werden i Vergleiche ausgeführt (danach ist j = 0). Für 6 Schützen: 1 + 2 + 3 + 4 + 5 = **15**. Für n Schützen: 1 + 2 + … + (n − 1) = **n(n − 1)/2**.

Beurteilung: Für n = 1000 sind es 499 500 Vergleiche, für n = 2000 sind es 1 999 000 – etwa viermal so viele. Da die Laufzeit im Wesentlichen von der Zahl der Vergleiche (und Vertauschungen) bestimmt wird, ist eher mit etwa 2 Sekunden zu rechnen. Die Aussage trifft für den ungünstigsten Fall nicht zu: Die Laufzeit wächst quadratisch, nicht linear.

Speicherbedarf: Die Operation sortieren tauscht innerhalb der Reihung teilnehmer und benötigt nur die Hilfsvariablen i, j und eine Tauschvariable – der zusätzliche Speicher ist konstant, unabhängig von n. Bei 2000 Schützen steigt also nur die Zahl der Vergleiche (etwa auf das Vierfache, ca. 2 s), nicht der Speicherbedarf.

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
11 Vergleiche bei Abbildung 3	I	1	☐
Ungünstigster Fall: aufsteigende Belegung angegeben, 15 Vergleiche	II	1	☐
Allgemeine Anzahl n(n − 1)/2 hergeleitet	II	1	☐
Beurteilung: Verdopplung → etwa vierfache Vergleichszahl → ca. 2 s; Aussage falsch; Speicherbedarf konstant (Tausch innerhalb der Reihung)	III	1	☐

Alternativen / Teilpunkte: Die Herleitung über die Summe 1 + 2 + … + (n − 1) oder über eine Tabelle ist gleichwertig. Eine Belegung mit gleichen Ringzahlen ist kein ungünstigster Fall (nur n − 1 Vergleiche). Zusatz (nicht gefordert): Im günstigsten Fall – bereits absteigend sortiert – sind es nur n − 1 Vergleiche; dann trifft die Verdopplung der Zeit tatsächlich ungefähr zu. Eine Argumentation mit $O(n^2)$ ohne Rechnung wird nur teilweise anerkannt, da die Begründung über die Zählung gefordert ist.

Typische Fehler

- Der Vergleich bei j = 0 wird mitgezählt (dann 13 statt 11 bzw. 20 statt 15).
- Absteigend sortierte Belegung als ungünstigster Fall (das ist der günstigste).
- „Doppelt so viele Schützen, doppelt so viele Vergleiche“.

d) Implementierung der Operation nachtragen 5 BE (I 1 · II 2 · III 2)

Musterlösung

Musterlösung (Java, mit javac und den Papier-Compiler-Tests geprüft: 8/8 grün):

```
public boolean nachtragen(Schuetze s) {
    if (s == null || anzahl == teilnehmer.length) {
        return false;
    }
    int i = anzahl;
    while (i > 0 && teilnehmer[i - 1].getRinge() < s.getRinge()) {
        teilnehmer[i] = teilnehmer[i - 1];
        i--;
    }
    teilnehmer[i] = s;
    anzahl++;
    return true;
}
```

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Methodenkopf; Abfrage null / voll mit Rückgabe false vor jeder Veränderung	I	1	☐
Verschieben schlechterer Schützen nach rechts, beginnend am Ende (kein Überschreiben)	II	1	☐
Vergleich über getRinge() mit echtem <, sodass s hinter gleich gute Schützen kommt	II	1	☐
Randfälle ohne Laufzeitfehler: i > 0 zuerst geprüft; Einfügen vorne, hinten und ins leere Turnier	III	1	☐
s einsetzen, anzahl erhöhen, true zurückgeben (Rückgabe in jedem Pfad)	III	1	☐

Alternativen / Teilpunkte: Gleichwertig (beide ebenfalls 8/8 Tests grün): (1) zuerst die Einfügeposition von vorne suchen (erster Index mit weniger Ringen als s), dann von hinten bis zu dieser Position verschieben; (2) s mit hinzufuegen(s) anhängen und anschließend wie in sortieren nach links tauschen (siehe unten). Struktogramm statt

Java: höchstens 3 BE (vgl. Abbildung EH.1). Kleinere Syntaxfehler führen nicht zu Abzug, wenn die Logik eindeutig ist – beim Papier-Compiler zählen sie aber!

Alternative (1): Position von vorne suchen, dann verschieben

```
public boolean nachtragen(Schuetze s) {
    if (s == null || anzahl == teilnehmer.length) {
        return false;
    }
    int pos = 0;
    while (pos < anzahl && teilnehmer[pos].getRinge() >= s.getRinge()) {
        pos++;
    }
    for (int k = anzahl; k > pos; k--) {
        teilnehmer[k] = teilnehmer[k - 1];
    }
    teilnehmer[pos] = s;
    anzahl++;
    return true;
}
```

Alternative (2): anhängen und nach links tauschen

```
public boolean nachtragen(Schuetze s) {
    if (!hinzufuegen(s)) {
        return false;
    }
    int j = anzahl - 1;
    while (j > 0 && teilnehmer[j - 1].getRinge() < teilnehmer[j].getRinge()) {
        Schuetze hilf = teilnehmer[j];
        teilnehmer[j] = teilnehmer[j - 1];
        teilnehmer[j - 1] = hilf;
        j--;
    }
    return true;
}
```

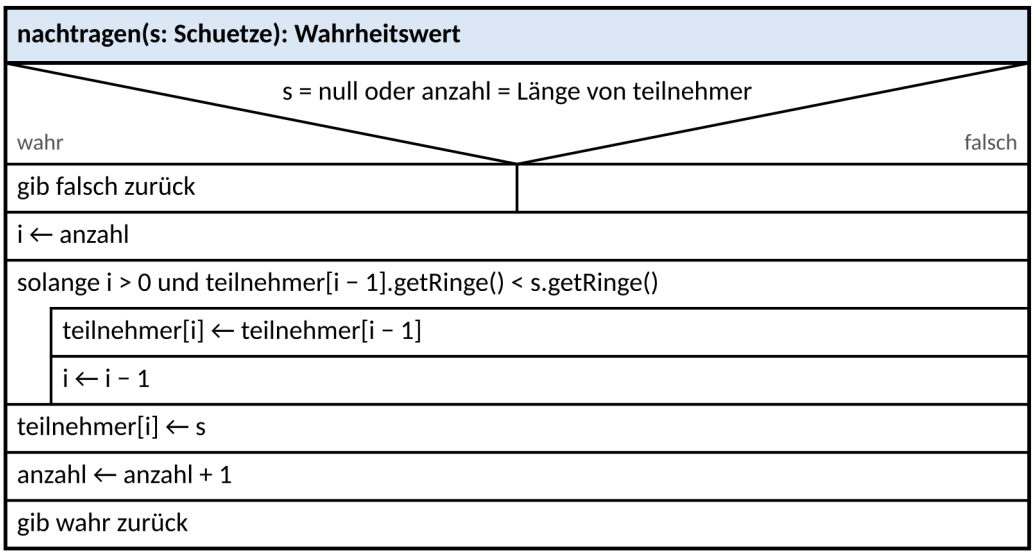


Abbildung EH.1: nachtragen als Struktogramm (bei Abgabe als Struktogramm höchstens 3 BE)

Typische Fehler

- <= statt < → s wird vor gleich gute Schützen gesetzt (Test gleichstandHintenAnstellen und mehrfachNachtragenBisVoll rot).
- Voll-Abfrage fehlt → ArrayIndexOutOfBoundsException bei vollem Turnier.
- i > 0 fehlt oder steht hinter dem Ringvergleich → Zugriff auf teilnehmer[−1], wenn s ganz nach vorne gehört.
- Verschieben von vorne nach hinten → Schützen werden überschrieben (Kopien desselben Objekts).
- anzahl++ vergessen oder Schleife bis teilnehmer.length (null-Felder).

e) Dokumentation des Werteverlaufs; Analyse der maximalen Durchläufe 4 BE (I 1 · II 3 · III 0)

Musterlösung

Werteverlauf (geprüft durch Ausführen einer instrumentierten Java-Fassung von Abbildung 5):

suche(nummern, 42)		
lo	hi	mid
0	14	7
8	14	11
8	10	9

suche(nummern, 20)		
lo	hi	mid
0	14	7
0	6	3
4	6	5
4	4	4

Rückgabewert suche(nummern, 42): **9** (nummern[9] = 42, 3 Durchläufe) · suche(nummern, 20): **−1** (4 Durchläufe, danach lo = 4 > hi = 3)

Analyse: In jedem Durchlauf wird nummern[mid] geprüft und – falls nr nicht gefunden wird – der Suchbereich auf die Elemente links oder rechts von mid verkleinert. Von k Elementen bleiben höchstens ⌊k/2⌋ übrig. Bei 100 Startnummern bleiben nach den Durchläufen höchstens 50, 25, 12, 6, 3, 1 Elemente; der 7. Durchlauf prüft das letzte verbliebene Element, danach ist der Bereich leer (lo > hi). Ein Aufruf benötigt also **höchstens 7 Schleifendurchläufe**; allgemein höchstens $\lfloor \log_2 n \rfloor + 1$.

Bei Verdopplung auf 200 Startnummern halbiert der erste Durchlauf den Bereich auf höchstens 100 – danach geht es wie oben weiter. Es ist also **genau ein Durchlauf mehr** nötig (höchstens 8). Zum Vergleich: Eine lineare Suche bräuchte bis zu 100 bzw. 200 Durchläufe.

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Werteverlauf und Rückgabewert 9 für suche(nummern, 42)	I	1	☐
Werteverlauf und Rückgabewert −1 für suche(nummern, 20)	II	1	☐
Begründung der höchstens 7 Durchläufe über die Halbierung des Suchbereichs	II	1	☐
Verdopplung: nur ein zusätzlicher Durchlauf (höchstens 8)	II	1	☐

Alternativen / Teilpunkte: Die Begründung über Zweierpotenzen ($2^7 = 128 \geq 100$) ist gleichwertig, sofern der Zusammenhang zur Halbierung hergestellt wird. Eine zusätzliche Zeile lo = 4, hi = 3 ohne mid nach dem letzten Durchlauf ist nicht fehlerhaft. Durch Ausprobieren aller 100 möglichen Positionen ermittelt: Maximum 7 (bei n = 200: 8).

Typische Fehler

- lo ← mid bzw. hi ← mid statt mid + 1 bzw. mid − 1 verwendet (falsche Werte, bei der Suche nach 20 Endlosschleife).
- Nach dem Treffer bei suche(nummern, 42) wird eine weitere Zeile protokolliert, obwohl die Operation mit „gib mid zurück“ endet.
- „100 : 2 : 2 … ≈ 6,6 → 6 Durchläufe“ (abgerundet).
- „Doppelt so viele Startnummern – doppelt so viele Durchläufe“.

f) Analyse des fehlerhaften Struktogramms 4 BE (I 1 · II 2 · III 1)

Musterlösung

Fehler 1: Startwert bester ← teilnehmer[0]. Der erste Schütze gehört nicht unbedingt zum gesuchten Verein. Hat er mindestens so viele Ringe wie alle Schützen des Vereins, wird ein vereinsfremder Schütze geliefert – z. B. nach dem Sortieren aus a) liefert besterAusVerein("Pfeilgilde") Tom (Eibenhain) statt Ben. Gibt es keinen Schützen des Vereins, wird nicht null geliefert. (Bei leerem Turnier ist teilnehmer[0] null; trifft dann eine Bedingung zu, scheitert bester.getRinge().)

Fehler 2: Die Zählschleife läuft bis anzahl. Im letzten Durchlauf ist teilnehmer[anzahl] null; der Aufruf getVerein() führt zu einem Laufzeitfehler (bei vollem Turnier liegt der Index außerhalb der Reihung). Der Fehler tritt bei jedem Aufruf auf, z. B. bei allen Aufrufen mit der Belegung aus Abbildung 3.

Fehler 3: Mit ≥ ersetzt ein späterer Schütze mit gleicher Ringzahl den bisherigen – geliefert wird der letzte statt der erste (kleinste Index). Korrekt ist >.

Korrigiertes Struktogramm: siehe Abbildung EH.2.

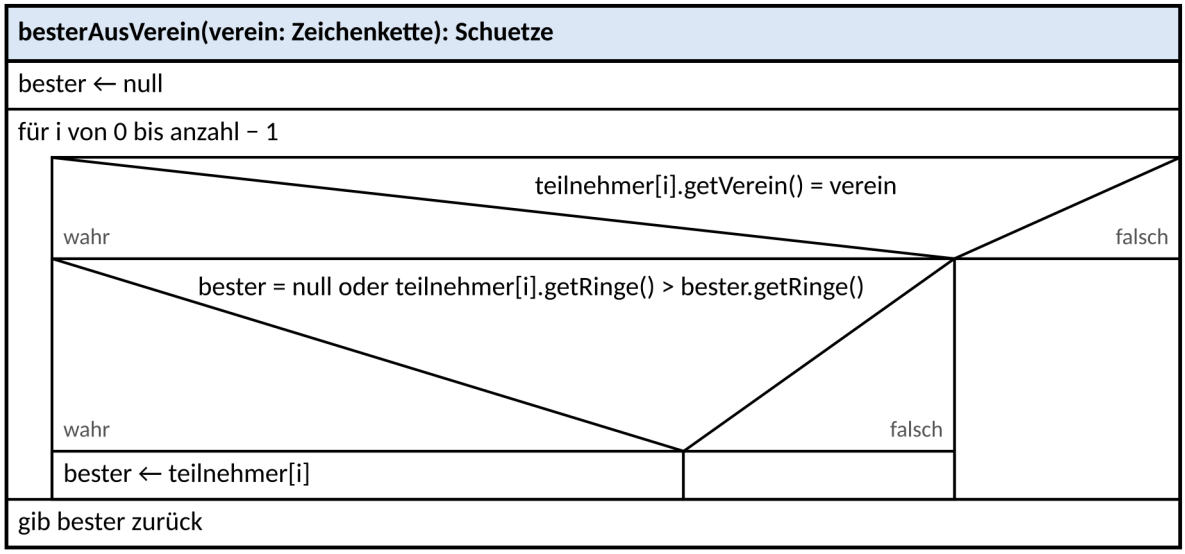


Abbildung EH.2: korrigiertes Struktogramm der Operation besterAusVerein

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Identifikation der drei Fehler	I	1	☐
Auswirkungen von Fehler 1 und Fehler 2	II	1	☐
Auswirkung von Fehler 3	II	1	☐
Vollständig korrigiertes Struktogramm (Notation nach EH §5)	III	1	☐

Alternativen / Teilpunkte: Gleichwertig: erst den ersten Schützen des Vereins suchen und ab dort vergleichen; Merken eines Index (−1 für „keiner“). Wird statt Fehler 1 der Start bei i = 1 als Fehler genannt und konsistent korrigiert (Start bei 0 mit bester ← null), ist das gleichwertig. Die Prüfung „bester = null“ muss vor dem Zugriff bester.getRinge() erfolgen (links in der oder-Verknüpfung oder in eigener Verzweigung).

Typische Fehler

- „≥ ist richtig, schließlich wird das Maximum gesucht“.
- Fehler 2 wird als „der letzte Schütze wird nicht geprüft“ beschrieben.
- Korrektur vergleicht bester.getRinge(), obwohl bester noch null sein kann.
- Korrigiertes Struktogramm ohne Kopf/Signatur oder mit = statt ←.

Anhang: Papier-Compiler-Workspace zu d)

Für den Papier-Compiler legst du in der Online-IDE einen Workspace „W2 Boss 2 – Papier-Compiler“ mit den Klassen Schuetze, Turnier (Methode nachtragen als leerer Rumpf mit return false;) und der Testklasse unten an und verteilst ihn über „An Klasse austeilen“. Die Lernenden tippen ihre Handschrift **unverändert** in den Rumpf ab und starten die Tests im Reiter „Testrunner“. Erst danach dürfen Fehler korrigiert werden (dann ohne Erster-Versuch-Bonus).

Vor Einsatz prüfen

JUnit-Dialekt der Online-IDE (parameterloser Konstruktor der Testklasse, @Test, globale assertEquals/assertTrue/assertFalse mit Meldung als letztem Argument) vor dem ersten Einsatz mit einem Probedurchlauf testen. Die Hilfsmethoden beispiel(…) und namen(…) der Testklasse sind private und ohne @Test – prüfen, dass der Testrunner sie nicht als Tests aufführt. Die Bibliothek „niedersachsen“ wird nicht benötigt.

Klasse Turnier (Gerüst für den Workspace)

```
public class Turnier {
    private Schuetze[] teilnehmer;
    private int anzahl;

    public Turnier(int plaetze) {
        teilnehmer = new Schuetze[plaetze];
        anzahl = 0;
    }

    public boolean hinzufuegen(Schuetze s) {
        if (s == null || anzahl == teilnehmer.length) {
            return false;
        }
        teilnehmer[anzahl] = s;
        anzahl++;
        return true;
    }

    public int getAnzahl() { return anzahl; }

    public Schuetze getSchuetze(int i) {
        if (i < 0 || i >= anzahl) {
            return null;
        }
        return teilnehmer[i];
    }

    public void sortieren() {
        for (int i = 1; i <= anzahl - 1; i++) {
            int j = i;
            while (j > 0 && teilnehmer[j - 1].getRinge() < teilnehmer[j].getRinge()) {
                Schuetze hilf = teilnehmer[j];
                teilnehmer[j] = teilnehmer[j - 1];
                teilnehmer[j - 1] = hilf;
                j = j - 1;
            }
        }
    }

    public boolean nachtragen(Schuetze s) {
        // HIER die Handschrift aus Aufgabe d) UNVERAENDERT abtippen
        return false;
```

```
    }
}
```

Klasse Schuetze (für den Workspace)

```
public class Schuetze {
    private String name;
    private String verein;
    private int ringe;

    public Schuetze(String name, String verein, int ringe) {
        this.name = name;
        this.verein = verein;
        this.ringe = ringe;
    }

    public String getName() { return name; }
    public String getVerein() { return verein; }
    public int getRinge() { return ringe; }
}
```

Testklasse BossTest (8 Tests; mit der Musterlösung und beiden Alternativen alle grün, typische Fehler werden erkannt – geprüft mit javac, siehe Tabelle)

```
public class BossTest {

    public BossTest() {
    }

    private Turnier beispiel(int plaetze) {
        // absteigend sortiert: Tom 58, Ben 52, Lea 52, Finn 50, Ida 47
        Turnier t = new Turnier(plaetze);
        t.hinzufuegen(new Schuetze("Tom", "Eibenhain", 58));
        t.hinzufuegen(new Schuetze("Ben", "Pfeilgilde", 52));
        t.hinzufuegen(new Schuetze("Lea", "Falkenhorst", 52));
        t.hinzufuegen(new Schuetze("Finn", "Pfeilgilde", 50));
        t.hinzufuegen(new Schuetze("Ida", "Falkenhorst", 47));
        return t;
    }

    private String namen(Turnier t) {
        String s = "";
        for (int i = 0; i < t.getAnzahl(); i++) {
            s = s + t.getSchuetze(i).getName() + " ";
        }
        return s;
    }

    @Test
    public void nullWirdAbgelehnt() {
        Turnier t = beispiel(8);
        assertFalse(t.nachtragen(null), "null: falsch erwartet");
        assertEquals(5, t.getAnzahl(), "null: anzahl muss 5 bleiben");
    }

    @Test
    public void vollesTurnierWirdAbgelehnt() {
        Turnier t = beispiel(5);
        assertFalse(t.nachtragen(new Schuetze("Jonas", "Eibenhain", 60)),
```

```
                "voll: falsch erwartet");
        assertEquals("Tom Ben Lea Finn Ida ", namen(t),
                "voll: Reihung darf sich nicht aendern");
    }

    @Test
    public void einfuegenInDerMitte() {
        Turnier t = beispiel(8);
        assertTrue(t.nachtragen(new Schuetze("Mila", "Falkenhorst", 49)),
                "wahr erwartet");
        assertEquals("Tom Ben Lea Finn Mila Ida ", namen(t),
                "Mila (49) gehoert zwischen Finn und Ida");
        assertEquals(6, t.getAnzahl(), "anzahl muss 6 sein");
    }

    @Test
    public void einfuegenGanzVorne() {
        Turnier t = beispiel(8);
        assertTrue(t.nachtragen(new Schuetze("Jonas", "Eibenhain", 60)),
                "wahr erwartet");
        assertEquals("Jonas Tom Ben Lea Finn Ida ", namen(t),
                "Jonas (60) gehoert an Index 0");
    }

    @Test
    public void einfuegenGanzHinten() {
        Turnier t = beispiel(8);
        assertTrue(t.nachtragen(new Schuetze("Pia", "Eibenhain", 30)),
                "wahr erwartet");
        assertEquals("Tom Ben Lea Finn Ida Pia ", namen(t),
                "Pia (30) gehoert ans Ende");
    }

    @Test
    public void gleichstandHintenAnstellen() {
        Turnier t = beispiel(8);
        t.nachtragen(new Schuetze("Jonas", "Eibenhain", 52));
        assertEquals("Tom Ben Lea Jonas Finn Ida ", namen(t),
                "Jonas (52) kommt HINTER Ben und Lea");
    }

    @Test
    public void leeresTurnier() {
        Turnier t = new Turnier(3);
        assertTrue(t.nachtragen(new Schuetze("Ida", "Falkenhorst", 47)),
                "wahr erwartet");
        assertEquals(1, t.getAnzahl(), "anzahl muss 1 sein");
        assertEquals("Ida", t.getSchuetze(0).getName(), "Ida muss an Index 0 stehen");
    }

    @Test
    public void mehrfachNachtragenBisVoll() {
        Turnier t = new Turnier(4);
        t.nachtragen(new Schuetze("A", "X", 40));
        t.nachtragen(new Schuetze("B", "X", 55));
        t.nachtragen(new Schuetze("C", "X", 40));
        assertTrue(t.nachtragen(new Schuetze("D", "X", 45)),
                "vierter Platz ist noch frei");
        assertFalse(t.nachtragen(new Schuetze("E", "X", 99)),
                "jetzt ist das Turnier voll");
```

```
        assertEquals("B D A C ", namen(t), "Reihenfolge B D A C erwartet");
    }
}
```

Kontrolle der Testklasse mit typischen Fehlvarianten (Mutanten)

Variante von nachtragen	Tests grün
Musterlösung (siehe d)	8/8
Alternative (1): Position von vorne suchen, dann verschieben	8/8
Alternative (2): anhängen und nach links tauschen	8/8
<= statt < (s vor gleich gute Schützen)	6/8
Voll-Abfrage fehlt	6/8
null-Abfrage fehlt	7/8
i > 0 fehlt (Zugriff auf teilnehmer[-1])	5/8
anzahl++ vergessen	2/8
Verschieben von vorne (überschreibt Schützen)	5/8
leerer Rumpf (Gerüst, return false;)	2/8